

SimpleRepeatSeq: clasificación de secuencias de ADN simples repetidas mediante aprendizaje automático libre de alineamiento

Carolina Quintero Valencia¹, Daniel Gomez Bohorquez¹, Isaac Enrique Castillo Devoz¹
¹ Maestría en Ingeniería Computacional, Universidad de Caldas, Manizales, Colombia

Resumen

Síntesis: SimpleRepeatSeq es una aplicación reproducible en Python para la clasificación binaria de secuencias de ADN como repeticiones simples (simpleSeq) o fragmentos genómicos no simples (no_simpleSeq). El sistema compara características libres de alineamiento basadas en frecuencias de k-meros y codificaciones numéricas de nucleótidos mediante clasificadores supervisados, proporcionando un flujo ejecutable para comparación de modelos y predicción a nivel de secuencia.

Disponibilidad e implementación: El código fuente, notebooks, modelos entrenados, reportes y figuras están disponibles en <https://github.com/carolinaqv/SimpleRepeatSeq>. La aplicación está implementada en Python y utiliza NumPy, pandas, Biopython, scikit-learn, XGBoost y Matplotlib. El archivo temporal del proyecto se encuentra disponible para los evaluadores del curso mediante la carpeta compartida entregada.

Contacto: iecastillod@uniguajira.edu.co

Información suplementaria: Las tablas completas de validación cruzada, 180 reportes de clasificación, curvas ROC, matrices de confusión y la interfaz de línea de comandos se incluyen en el repositorio.

Palabras clave: repeticiones cortas en tándem; secuencias simples repetidas; k-meros; clasificación libre de alineamiento; aprendizaje automático; bioinformática.

1 Introducción

Las repeticiones cortas en tándem (STR), también denominadas secuencias simples repetidas o microsatélites, son intervalos genómicos compuestos por motivos cortos repetidos consecutivamente. Su variabilidad de longitud contribuye a la evolución genómica, la variación poblacional y la identificación forense; además, expansiones en motivos como CAG, CGG, CTG, GAA y GGGGCC participan en trastornos neurológicos y neuromusculares [1,2]. Debido a que la composición repetitiva puede afectar el alineamiento de lecturas y el ensamblaje, el cribado computacional robusto de fragmentos con apariencia repetitiva sigue siendo relevante en bioinformática genómica.

Las herramientas consolidadas abordan el análisis de repeticiones mediante detección y enmascaramiento especializados. Tandem Repeats Finder identifica arreglos en tándem directamente desde secuencias de ADN sin requerir un motivo preespecificado [3], mientras que RepeatMasker detecta repeticiones intercaladas y regiones de baja complejidad mediante bibliotecas de elementos repetitivos y flujos de anotación [4,5]. Estos programas son centrales en la anotación genómica, pero no están diseñados principalmente como clasificadores supervisados ligeros para fragmentos cortos y ruidosos de ADN.

SimpleRepeatSeq complementa la anotación clásica formulando el reconocimiento de repeticiones como un problema binario de clasificación libre de alineamiento. La aplicación convierte cadenas de ADN en vectores de longitud fija mediante frecuencias de k-meros o codificaciones numéricas, y evalúa clasificadores supervisados bajo un protocolo estandarizado. Las representaciones basadas en palabras son ampliamente usadas para comparación y clasificación de secuencias porque capturan composición local sin alineamiento por pares [6,7].

2 Sistema y métodos

2.1 Dataset y diseño de validación

El benchmark utilizó SimpleRepeatSeq_dataset.fasta, un corpus balanceado de 888 secuencias de ADN: 444 registros simpleSeq y 444 registros no_simpleSeq. Las longitudes oscilaron entre 100 y 500 pb e incluyeron ruido de secuenciación simulado entre 4% y 12%. Los identificadores FASTA siguieron el formato nombre_secuencia#clase#longitud. Las etiquetas se emplearon únicamente para entrenamiento y evaluación supervisada; las características se calcularon exclusivamente a partir del contenido nucleotídico.

La clase positiva incluyó repeticiones teloméricas, motivos asociados con expansiones patológicas, STR forenses, satélites centroméricos y microsatélites di- y trinucleotídicos. La clase negativa incluyó exones, intrones, promotores CpG, regiones no traducidas, enhancers, regiones mitocondriales y fragmentos tipo SINE. El rendimiento se estimó mediante

conjunto externo independiente; por tanto, los resultados deben interpretarse como desempeño interno del benchmark y no como evidencia de generalización universal para despliegue.

2.2 Codificaciones, preprocesamiento y modelos

Se implementaron seis codificaciones. Los vectores de frecuencia normalizada de k-meros se generaron para k=3, k=4 y k=6 usando el alfabeto {A,C,G,T,N}, lo que produjo 125, 625 y 15.625 características, respectivamente. Los k-meros se extrajeron con ventana deslizante de paso unitario y se normalizaron por el número de ventanas observadas. También se evaluaron tres codificaciones numéricas: DAX (A=2, C=0, G=3, T=1, N=1.5), EIIP o potencial de interacción electrón-ion (A=0.126, C=0.134, G=0.081, T=0.134) y una escala complementaria purina/pirimidina (A=2, C=-1, G=1, T=-2).

Para cada codificación se probaron tres estrategias de preprocesamiento: características crudas, estandarización z-score con StandardScaler y estandarización seguida de análisis de componentes principales reteniendo al menos 96% de la varianza acumulada. El preprocesamiento se integró dentro del flujo de validación cruzada para reducir fuga de información, siguiendo convenciones de scikit-learn [8]. Se compararon diez clasificadores: Gaussian Naive Bayes, máquina de soporte vectorial, random forest, gradient boosting classifier, XGBoost, perceptrón multicapa, árbol de decisión, k vecinos más cercanos, regresión logística y análisis discriminante lineal. XGBoost se incluyó como una línea base robusta de árboles potenciados por gradiente [9], y el perceptrón multicapa como comparador de red neuronal completamente conectada [10].

Se reportaron accuracy, precisión, recall, F1 ponderado y ROC-AUC macro. ROC-AUC se empleó porque resume la discriminación binaria dependiente del umbral [11]. El procesamiento de secuencias y la visualización usaron Biopython y Matplotlib [12,13].

3 Resultados

La cuadrícula completa comprendió 180 configuraciones: seis codificaciones, tres estrategias de preprocesamiento y diez clasificadores. Las representaciones por k-meros dominaron el benchmark. El ROC-AUC promedio por codificación fue 0.9891 para k=6, 0.9890 para k=4 y 0.9889 para k=3; en contraste, las codificaciones numéricas mostraron menor desempeño promedio: 0.9062 para EIIP, 0.8823 para la codificación complementaria y 0.8542 para DAX.

Cuarenta y seis configuraciones alcanzaron ROC-AUC=1.0000, todas basadas en k-meros. PCA obtuvo el mayor ROC-AUC promedio entre todas las configuraciones (0.9614), pero los mejores modelos individuales no requirieron reducción de dimensionalidad. Los vectores crudos de k-meros fueron suficientes para obtener desempeño perfecto de validación cruzada interna con varios clasificadores.

Validación cruzada estratificada de 10 particiones. No se dispuso de un

Tabla 1. Configuraciones representativas de mejor desempeño de SimpleRepeatSeq bajo validación cruzada estratificada de 10 particiones.

Codificación	Preproc.	Modelo	Accuracy	Precisión	Recall	F1	ROC-AUC
k-mero k=3	Crudo	Naive Bayes	1.0000	1.0000	1.0000	1.0000	1.0000
k-mero k=4	Crudo	Naive Bayes	1.0000	1.0000	1.0000	1.0000	1.0000
k-mero k=6	Crudo	Naive Bayes	1.0000	1.0000	1.0000	1.0000	1.0000
Complementaria	Crudo	SVM	0.9899	0.9899	0.9899	0.9899	0.9996
DAX	Crudo	XGBoost	0.9876	0.9877	0.9876	0.9876	0.9995
EIIP	Escalado	MLP	0.9854	0.9854	0.9854	0.9854	0.9993

La implementación seleccionada fue frecuencias crudas de k-meros k=3 con Gaussian Naive Bayes. Este modelo alcanzó accuracy=1.0000, precisión=1.0000, recall=1.0000, F1 ponderado=1.0000 y ROC-AUC=1.0000, clasificando correctamente 444 registros no_simpleSeq y 444 registros simpleSeq en el benchmark interno.

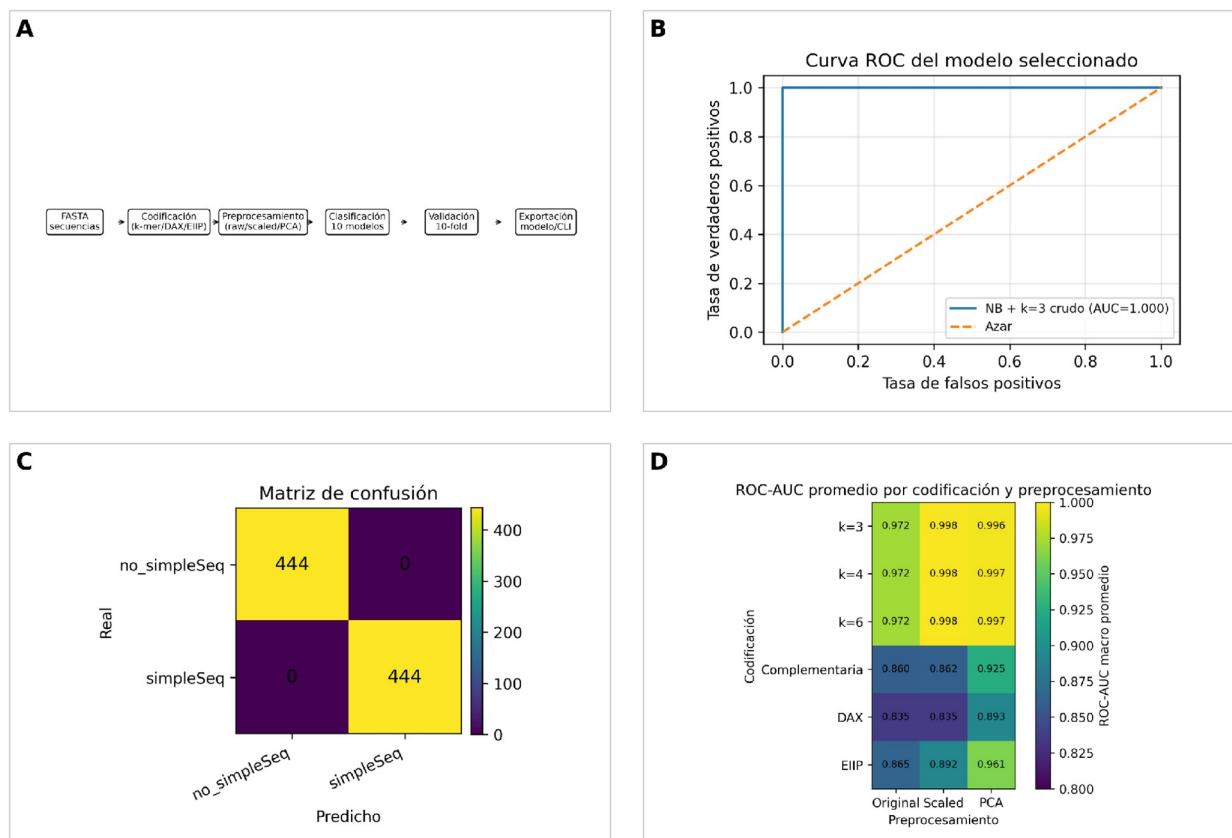


Fig. 1. Resumen del benchmark SimpleRepeatSeq. (A) Flujo completo desde lectura FASTA hasta exportación del modelo. (B) Curva ROC del clasificador Naive Bayes seleccionado con frecuencias crudas de k-meros k=3. (C) Matriz de confusión del modelo seleccionado. (D) ROC-AUC promedio por codificación y estrategia de preprocesamiento, mostrando la robustez de las representaciones por k-meros.

4 Discusión

El predominio de las codificaciones por k-meros es biológicamente coherente: las repeticiones simples se definen por motivos cortos recurrentes, y las frecuencias de k-meros capturan directamente la composición local de palabras nucleotídicas. Aunque k=6 puede codificar patrones hexanucleotídicos, k=3 fue seleccionado para el modelo final porque obtuvo desempeño idéntico con solo 125 características, sin escalado ni PCA. Esta parsimonia mejora la interpretabilidad, el tiempo de ejecución y la idoneidad para despliegue por línea de comandos.

Las codificaciones numéricas también alcanzaron valores máximos altos de ROC-AUC cuando se combinaron con clasificadores flexibles como SVM, XGBoost y MLP, lo que indica que las escalas fisicoquímicas a nivel de base pueden conservar señales útiles de repetición. Sin embargo, su menor desempeño promedio sugiere mayor sensibilidad al preprocesamiento y al clasificador seleccionado.

La principal limitación es que el corpus es controlado, balanceado y parcialmente simulado. Los datos genómicos reales suelen ser desbalanceados e incluyen intervalos homólogos, repeticiones anidadas, artefactos de secuenciación y regiones de baja complejidad no repetitivas. Por tanto, el desempeño perfecto en validación cruzada debe reportarse transparentemente como resultado interno. Trabajos futuros deberían evaluar intervalos genómicos externos, añadir separación entrenamiento/prueba consciente de homología, comparar directamente con TRF y RepeatMasker, y extender el sistema hacia identificación de motivos, estimación de límites de repetición y predicción multiclase por familia repetitiva.

5 Disponibilidad e implementación

SimpleRepeatSeq está implementado en Python y se distribuye con notebooks, modelos entrenados, figuras, reportes de clasificación e interfaz de línea de comandos. Repositorio: <https://github.com/carolinaqv/SimpleRepeatSeq>. Uso sugerido de la CLI: `python classify.py --fasta input.fasta --model models/km3_Original_NB.pkl --k 3 --encoding km3`. El programa retorna identificador de secuencia, clase predicha y probabilidad de predicción.

Referencias

- Gymrek M. A genomic view of short tandem repeats. *Curr Opin Genet Dev.* 2017;44:9-16. doi:10.1016/j.gde.2017.01.012.

- Malik I, Kelley CP, Wang ET, Todd PK. Molecular mechanisms underlying nucleotide repeat expansion disorders. *Nat Rev Mol Cell Biol.* 2021;22:589-607. doi:10.1038/s41580-021-00382-6.
- Benson G. Tandem repeats finder: a program to analyze DNA sequences. *Nucleic Acids Res.* 1999;27(2):573-580. doi:10.1093/nar/27.2.573.
- Smit AFA, Hubley R, Green P. RepeatMasker Open-4.0. 2013-2015. Available from: <http://www.repeatmasker.org>.
- Tarailo-Graovac M, Chen N. Using RepeatMasker to identify repetitive elements in genomic sequences. *Curr Protoc Bioinformatics.* 2009;25:4.10.1-4.10.14. doi:10.1002/0471250953.bi0410s25.
- Vinga S, Almeida J. Alignment-free sequence comparison-a review. *Bioinformatics.* 2003;19(4):513-523. doi:10.1093/bioinformatics/btg005.
- Leslie C, Eskin E, Noble WS. The spectrum kernel: a string kernel for SVM protein classification. *Pac Symp Biocomput.* 2002;7:564-575.
- Pedregosa F, Varoquaux G, Gramfort A, Michel V, Thirion B, Grisel O, et al. Scikit-learn: machine learning in Python. *J Mach Learn Res.* 2011;12:2825-2830.
- Chen T, Guestrin C. XGBoost: a scalable tree boosting system. In: *Proceedings of the 22nd ACM SIGKDD International Conference on Knowledge Discovery and Data Mining.* 2016. p. 785-794. doi:10.1145/2939672.2939785.
- LeCun Y, Bengio Y, Hinton G. Deep learning. *Nature.* 2015;521:436-444. doi:10.1038/nature14539.
- Fawcett T. An introduction to ROC analysis. *Pattern Recognit Lett.* 2006;27(8):861-874. doi:10.1016/j.patrec.2005.10.010.
- Cock PJA, Antao T, Chang JT, Chapman BA, Cox CJ, Dalke A, et al. Biopython: freely available Python tools for computational molecular biology and bioinformatics. *Bioinformatics.* 2009;25(11):1422-1423. doi:10.1093/bioinformatics/btp163.
- Hunter JD. Matplotlib: a 2D graphics environment. *Comput Sci Eng.* 2007;9(3):90-95. doi:10.1109/MCSE.2007.55.